UNIDAD 7 • REACT ROUTER II • METODOLOGÍA PASO A PASO

🏠 Manual Práctico: Proyecto "InmoPortal"

Catálogo de Propiedades Inmobiliarias con Enfoque Iterativo, Renderizado Inmediato y Explicación Línea por Línea ("Receta de Cocina")

📌 Introducción y Objetivos del Ejercicio

Este manual documenta paso a paso la construcción desde cero de **InmoPortal**, una aplicación de una sola página (SPA) aplicada a la vida real para consultar fichas técnicas de propiedades inmobiliarias y administrar el acceso privado de corredores.

Los 4 objetivos fundamentales de React Router II que dominamos en este proyecto son:

1. **Componente `<NavLink>` y propiedad `isActive`:** Crear un menú de navegación que detecte automáticamente la ruta activa y le aplique estilos visuales distintos (`.active` vs `.inactive`).
2. **Hook `useNavigate()`:** Redireccionar programáticamente al usuario desde una función tras elegir una propiedad en un selector y validar su selección.
3. **Hook `useParams()`:** Capturar parámetros dinámicos de la URL (`/propiedades/:id`) para buscar la propiedad en nuestro catálogo y mostrarla en una tarjeta `Card` de React Bootstrap.
4. **Rutas Protegidas con `UserContext` y `<Navigate />`:** Restringir el acceso a la ruta privada `/admin` evaluando si existe una sesión activa, redirigiendo automáticamente a `/login` en caso contrario.

🗺️ Hoja de Ruta General (Enfoque de Renderizado Inmediato)

Paso	Qué construimos	Qué comprobamos en pantalla
Paso 1	Creación del proyecto con Vite, instalación de `react-router-dom` y limpieza de `App.jsx`.	Título inicial controlado por nosotros en `http://localhost:5173`.
Paso 2	Instalación de Bootstrap / React-Bootstrap y estilos `.active` e `.inactive` en `index.css`.	Tipografía limpia de Bootstrap y texto centrado en pantalla.
Paso 3	Componente `Navbar.jsx` con `NavLink` e importación en `App.jsx` dentro de `BrowserRouter`.	Barra de navegación superior cambiando de color verde (activo) a gris (inactivo) al hacer clic.
Paso 4	Vista `Home.jsx` con selector de propiedades, validación y hook `useNavigate()`.	Alerta si no se elige propiedad, y cambio de la URL a `/propiedades/1` al buscar.
Paso 5	Archivo `data/propiedades.js` y vista `Propiedades.jsx` con `useParams()` y `Card` de Bootstrap.	Primero el número `#id` en pantalla y luego la tarjeta con foto, precio en UF y alerta amarilla si no existe.
Paso 6	Vistas `Administracion.jsx` y `Login.jsx`, `UserContext.jsx` y ruta protegida con `<Navigate />`.	Al hacer clic en "Administración" sin sesión activa, el guardia nos rebota automáticamente a `/login`.
Paso 7	Formulario real de Login (Nombre y Correo) y botón de Cerrar Sesión en `Administracion.jsx`.	Ingreso con tu propio nombre ("Bienvenido Jorge Leiva") y expulsión automática al cerrar sesión.

📁 Estructura Final de Archivos del Proyecto

```
inmo-portal/
├── src/
│   ├── components/
│   │   └── Navbar.jsx          # Barra de navegación con NavLink e isActive
│   ├── context/
│   │   └── UserContext.jsx     # Estado global de sesión (user y setUser)
│   ├── data/
│   │   └── propiedades.js      # Arreglo de objetos con las 5 propiedades inmobiliarias
│   ├── views/
│   │   ├── Home.jsx            # Buscador con selector y redirección con useNavigate()
│   │   ├── Propiedades.jsx     # Vista detalle con useParams() y Card de React Bootstrap
│   │   ├── Administracion.jsx  # Vista privada exclusiva para corredores con sesión activ
│   │   └── Login.jsx           # Formulario de inicio de sesión conectado a UserContext
│   ├── App.jsx                 # Enrutador BrowserRouter, Routes, Route y guardia Navigat
│   ├── index.css               # Estilos generales y clases .active / .inactive
│   └── main.jsx                # Punto de entrada envolviendo App con UserProvider
```

PASO 1 Crear el Proyecto Base e Instalar React Router DOM

Antes de crear vistas o componentes, preparamos el proyecto con **Vite**, instalamos la librería de enrutamiento y limpiamos el archivo `App.jsx` para tener nuestra primera vista controlada en pantalla.

1.1. Comandos en la terminal

```
npm create vite@latest inmo-portal -- --template react
cd inmo-portal
npm install
npm install react-router-dom
npm run dev
```

🔍 **Explicación comando por comando ("Receta de cocina"):**

- `npm create vite@latest inmo-portal -- --template react` : Crea la carpeta `inmo-portal` con la plantilla oficial de React en JavaScript.
- `cd inmo-portal` : Nos ubica dentro de la carpeta del proyecto recién creado.
- `npm install` : Descarga todas las dependencias base de React dentro de `node_modules` .
- `npm install react-router-dom` : Instala la librería React Router DOM que nos provee `BrowserRouter` , `Routes` , `Route` , `NavLink` , `useNavigate` , `useParams` y `Navigate` .
- `npm run dev` : Enciende el servidor local de desarrollo (por defecto en `http://localhost:5173`).

1.2. Limpiar `src/App.jsx` para nuestro primer renderizado

📄 **src/App.jsx (Versión Paso 1)**

```
export default function App() {
  return (
    <div className="App">
      <h1>🏠 InmoPortal - Catálogo de Propiedades</h1>
      <p>Paso 1 completado: Nuestro proyecto base está funcionando.</p>
    </div>
  );
}
```

🔍 **Explicación línea por línea:**

- `export default function App()`: Define el componente raíz `App` y lo exporta por defecto para que `main.jsx` pueda inyectarlo en el HTML.
- `<div className="App">`: Contenedor principal. En JSX usamos `className` en lugar de `class` porque `class` es una palabra reservada de JavaScript.

✅ **Punto de control en pantalla (Paso 1):**

Al abrir `http://localhost:5173` en el navegador, vemos el título "🏠 **InmoPortal - Catálogo de Propiedades**".

PASO 2 Instalar Bootstrap y Configurar `index.css`

Instalamos **Bootstrap** (hoja de estilos CSS) y **React-Bootstrap** (componentes de React como `Card` y `Badge`), y definimos en `index.css` las clases para estilizar la ruta activa (`.active`) y las rutas inactivas (`.inactive`).

2.1. Instalar Bootstrap en la terminal

```
npm install react-bootstrap bootstrap
```

2.2. Importar el CSS de Bootstrap en `src/main.jsx`

📄 **src/main.jsx (Versión Paso 2)**

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import 'bootstrap/dist/css/bootstrap.min.css'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

🔍 **¿Por qué `bootstrap.min.css` se importa ANTES que `./index.css`?**

En CSS el orden importa ("cascada"). Al cargar primero Bootstrap y debajo nuestro `./index.css`, nos aseguramos de que nuestras clases personalizadas tengan prioridad y no sean sobrescritas por Bootstrap.

2.3. Escribir los estilos de navegación en `src/index.css`

📄 **src/index.css**

```
.App {
  font-family: sans-serif;
  text-align: center;
}

nav {
  display: flex;
  justify-content: center;
  gap: 20px;
  padding: 20px;
  background-color: #f8f9fa;
  border-bottom: 1px solid #dee2e6;
}

.active {
  color: darkgreen;
  font-weight: bold;
  text-decoration: underline;
}

.inactive {
  color: #6c757d;
  text-decoration: none;
}
```

🔍 **Explicación de cada regla CSS:**

- `.App` : Aplica tipografía limpia y centra todo el texto de nuestra aplicación.
- `nav` : Usa Flexbox (`display: flex; justify-content: center; gap: 20px;`) para alinear los enlaces del menú horizontalmente en el centro con 20px de separación entre cada opción.
- `.active` : Pinta de color **verde oscuro** (`darkgreen`), en negrita y subrayado el enlace que coincida con la ruta actual del navegador.
- `.inactive` : Pinta de color **gris suave** (`#6c757d`) y sin subrayado los enlaces de las rutas que NO están siendo visitadas en ese momento.

PASO 3 Componente `Navbar.jsx` con `NavLink` y Renderizado

A diferencia de `<Link>`, el componente **`<NavLink>`** sabe automáticamente si la ruta de su atributo `to` está siendo visitada gracias a su propiedad interna booleana **`isActive`** (`true` o `false`).

3.1. Crear `src/components/Navbar.jsx`

📄 **src/components/Navbar.jsx**

```
import { NavLink } from 'react-router-dom';

export default function Navbar() {
  const setActiveClass = ({ isActive }) => (isActive ? 'active' : 'inactive');

  return (
    <nav>
      <NavLink className={setActiveClass} to='/'>
        Home
      </NavLink>
      <NavLink className={setActiveClass} to='/propiedades'>
        Propiedades
      </NavLink>
      <NavLink className={setActiveClass} to='/admin'>
        Administración
      </NavLink>
      <NavLink className={setActiveClass} to='/login'>
        Login
      </NavLink>
    </nav>
  );
}
```

🔍 **Explicación línea por línea de `Navbar.jsx`:**

- `import { NavLink } from 'react-router-dom';`: Extrae el componente `NavLink` desde la librería (lleva llaves `{ }` porque es una exportación nombrada de React Router).
- `const setActiveClass = ({ isActive }) => (isActive ? 'active' : 'inactive');`: Función evaluadora que recibe el objeto interno de `NavLink`, desestructura la propiedad `{ isActive }` y usa un operador ternario: si `isActive` es `true` retorna la clase `'active'`; si es `false` retorna `'inactive'`.
- `<NavLink className={setActiveClass} to='/...'>`: Conecta nuestra función al atributo `className` y define la ruta de destino en `to`.

3.2. Importar y renderizar `Navbar` en `src/App.jsx`

📄 **src/App.jsx (Versión Paso 3)**

```
import { BrowserRouter } from 'react-router-dom';
import Navbar from './components/Navbar';

function App() {
  return (
    <div className='App'>
      <BrowserRouter>
        <Navbar />
        <h1>🏠 InmoPortal - Catálogo de Propiedades</h1>
        <p>Paso 3: Probando el componente Navbar con NavLink</p>
      </BrowserRouter>
    </div>
  )
}

export default App
```

⚠️ **Errores comunes que debemos evitar en este paso (Lecciones aprendidas):**

1. **No usar llaves `{ }` al importar `Navbar`:** Como en `Navbar.jsx` usamos `export default function Navbar()`, en `App.jsx` se debe importar como `import Navbar from './components/Navbar';` (sin llaves). Si pones `import { Navbar }`, la pantalla queda en blanco.
2. **`<Navbar />` SIEMPRE debe ir ADENTRO de `<BrowserRouter>`:** Si colocas `<Navbar />` una línea antes de abrir `<BrowserRouter>`, los `<NavLink>` fallarán porque necesitan estar envueltos por el enrutador para leer la URL.

✅ **Punto de control en pantalla (Paso 3):**

Al hacer clic en **Home**, **Propiedades**, **Administración** o **Login**, la URL cambia arriba sin recargar la página y la opción seleccionada se pinta de verde oscuro y subrayada.

PASO 4 Redireccionamiento Programático con `useNavigate()` en `Home.jsx`

El **redireccionamiento programático** ocurre cuando necesitamos llevar al usuario a otra ruta desde una función de JavaScript (por ejemplo, después de validar que haya seleccionado una propiedad en un desplegable `<select>`) e inyectar una variable en la URL de destino (`/propiedades/${id}`).

4.1. Crear la vista `src/views/Home.jsx`

src/views/Home.jsx

```
import { useState } from "react";
import { useNavigate } from 'react-router-dom';

export default function Home() {
  const [id, setId] = useState('');
  const navigate = useNavigate();

  const irAPropiedades = () => {
    if (id === '') {
      alert('Por favor, selecciona una propiedad antes de buscar.');
      return;
    }
    navigate(`/propiedades/${id}`);
  };

  return (
    <div className="mt-5 container">
      <h1>🏡 Encuentra tu Propiedad Ideal</h1>
      <p className="text-muted">
        Selecciona el código de la propiedad para ver su ficha técnica
      </p>

      <div className="d-flex justify-content-center gap-2 mt-4">
        <select
          className="form-select w-auto"
          value={id}
          onChange={({ target }) => setId(target.value)}
        >
          <option value="">-- Selecciona una propiedad (1 al 5) --</option>
          <option value="1">Propiedad #1 – Casa Mediterránea (Las Condes)</option>
          <option value="2">Propiedad #2 – Depto Loft (Providencia)</option>
          <option value="3">Propiedad #3 – Parcela con Piscina (Chicureo)</option>
          <option value="4">Propiedad #4 – Penthouse Vista al Mar (Viña del Mar)</option>
          <option value="5">Propiedad #5 – Cabaña Bosque Nativo (Pucón)</option>
        </select>

        <button className="btn btn-success" onClick={irAPropiedades}>
          Buscar Propiedad
        </button>
      </div>
    </div>
  );
}
```

🔍 **Explicación línea por línea de `Home.jsx`:**

- `const [id, setId] = useState('');`: Crea el estado `id` inicialmente vacío para guardar qué opción eligió el usuario en el selector.
- `const navigate = useNavigate();`: Inicializa el hook `useNavigate` de React Router y nos devuelve la función `navigate`.
- `if (id === '') { alert(...); return; }`: Condiciona el redireccionamiento. Si el usuario no ha elegido ninguna propiedad, muestra una alerta y el `return` frena la ejecución.
- `navigate(`/propiedades/${id}`);`: Usa comillas invertidas (template literals) para redirigir programáticamente al usuario hacia la URL con el código elegido (ej: `/propiedades/3`).
- `onChange={({ target }) => setId(target.value)}`: Captura el valor seleccionado en el `<select>` y lo guarda en el estado mediante `setId`.

⚠️ **Error común que resolvimos en `Home.jsx`:**

En el atributo `onChange` del `<select>`, nunca olvides escribir **`setId`** antes de `(target.value)`. Si escribes solo `onChange={({ target }) => (target.value)}` sin llamar a `setId`, el estado `id` quedará vacío para siempre y el selector no cambiará de opción.

4.2. Conectar la ruta `/` en `src/App.jsx`

📄 **src/App.jsx (Versión Paso 4)**

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Home from './views/Home';

function App() {
  return (
    <div className='App'>
      <BrowserRouter>
        <Navbar />
        <Routes>
          <Route path='/' element={<Home />} />
        </Routes>
      </BrowserRouter>
    </div>
  )
}

export default App
```

✅ **Punto de control en pantalla (Paso 4):**

Al elegir la **Propiedad #1** en el desplegable y presionar **"Buscar Propiedad"**, la barra de direcciones cambia a `http://localhost:5173/propiedades/1` y en el menú superior se ilumina **"Propiedades"** en verde automáticamente.

PASO 5 Capturar Parámetros con `useParams()` y Mostrar Tarjeta `Card`

Cuando definimos una ruta con dos puntos como `path='/propiedades/:id'`, React Router reconoce que todo lo que venga después de `/propiedades/` es un parámetro dinámico llamado `id`. Para leer ese valor dentro del componente destino, utilizamos el hook **`useParams()`**.

5.1. Primera prueba de `useParams()` en `src/views/Propiedades.jsx`

Primero creamos una versión mínima de `Propiedades.jsx` y la conectamos en `App.jsx` para verificar en pantalla que atrapamos el número de la URL:

`src/views/Propiedades.jsx (Versión 5.1 – Prueba de captura de ID)`

```
import { useParams } from 'react-router-dom';

export default function Propiedades() {
  const { id } = useParams();

  return (
    <div className='mt-5'>
      <h2>Ficha de la Propiedad</h2>
      <h1>Código recibido por la URL: #{id}</h1>
    </div>
  )
}
```

> ⚠️ **Dos detalles clave que verificamos en este sub-paso:**
>
> 1. **No olvidar `const { id } = useParams();` antes del `return`:** Si importas `useParams` en la línea 1 pero olvidas ejecutarlo dentro de la función, la variable `{id}` no existirá.
> 2. **No duplicar la etiqueta de cierre `</Routes>` en `App.jsx`:** Cada `<Routes>` que se abre debe tener un único cierre `</Routes>` antes de `</BrowserRouter>`.

5.2. Separar la base de datos en `src/data/propiedades.js`

Como buena práctica profesional, guardamos el catálogo de propiedades en una carpeta separada `src/data` y lo exportamos con `export const`:

src/data/propiedades.js

```
export const catalogoPropiedades = [
  {
    id: "1",
    titulo: "Casa Mediterránea en Las Condes",
    ubicacion: "Las Condes, Santiago",
    precio: "14.500 UF",
    habitaciones: "4 Dormitorios | 3 Baños | 280 m²",
    imagen: "https://images.unsplash.com/photo-1600596542815-ffad4c1539a9?auto=format&fit=c
  },
  {
    id: "2",
    titulo: "Depto Loft Moderno en Providencia",
    ubicacion: "Providencia, Santiago",
    precio: "6.800 UF",
    habitaciones: "2 Dormitorios | 2 Baños | 95 m²",
    imagen: "https://images.unsplash.com/photo-1522708323590-d24dbb6b0267?auto=format&fit=c
  },
  {
    id: "3",
    titulo: "Parcela con Piscina en Chicureo",
    ubicacion: "Chicureo, Colina",
    precio: "18.200 UF",
    habitaciones: "5 Dormitorios | 4 Baños | 5.000 m²",
    imagen: "https://images.unsplash.com/photo-1600585154340-be6161a56a0c?auto=format&fit=c
  },
  {
    id: "4",
    titulo: "Penthouse Vista al Mar en Viña del Mar",
    ubicacion: "Reñaca, Viña del Mar",
    precio: "11.900 UF",
    habitaciones: "3 Dormitorios | 3 Baños | 160 m²",
    imagen: "https://images.unsplash.com/photo-1512917774080-9991f1c4c750?auto=format&fit=c
  },
  {
    id: "5",
    titulo: "Cabaña Bosque Nativo en Pucón",
    ubicacion: "Camino al Volcán, Pucón",
    precio: "7.400 UF",
    habitaciones: "3 Dormitorios | 2 Baños | 1.500 m²",
    imagen: "https://images.unsplash.com/photo-1518780664697-55e3ad937233?auto=format&fit=c
  },
];
```

5.3. Mostrar la tarjeta `Card` condicionada en `src/views/Propiedades.jsx`

📄 **src/views/Propiedades.jsx (Versión Definitiva con Card y Validación)**

```
import { useParams } from "react-router-dom";
import { Card, Badge } from "react-bootstrap";
import { catalogoPropiedades } from "../data/propiedades";

export default function Propiedades() {
  const { id } = useParams();

  const propiedad = catalogoPropiedades.find((item) => item.id === id);

  return (
    <div className="mt-4 container d-flex justify-content-center">
      {propiedad ? (
        <Card style={{ width: "24rem" }} className="shadow-sm">
          <Card.Img variant="top" src={propiedad.imagen} alt={propiedad.titulo} />
          <Card.Body>
            <Badge bg="success" className="mb-2">
              Código #{propiedad.id} - {propiedad.precio}
            </Badge>
            <Card.Title>{propiedad.titulo}</Card.Title>
            <Card.Text className="text-muted mb-1">
              📍 {propiedad.ubicacion}
            </Card.Text>
            <Card.Text className="fw-bold">
              🛏️ {propiedad.habitaciones}
            </Card.Text>
          </Card.Body>
        </Card>
      ) : (
        <div className="alert alert-warning mt-4">
          <h4>⚠️ Propiedad no encontrada</h4>
          <p className="mb-0">
            No se encontró ninguna propiedad con el código <strong>#{id || "sin especificar
            Por favor, vuelve al Home y selecciona una propiedad válida.
          </p>
        </div>
      )}
    </div>
  );
}
```

🔍 **Explicación línea por línea de `Propiedades.jsx`:**

- `import { catalogoPropiedades } from "../data/propiedades";`: Sube un nivel con `..` desde `views` hacia `src` y extrae con llaves `{ catalogoPropiedades }` nuestro arreglo de datos.
- `const propiedad = catalogoPropiedades.find((item) => item.id === id);`: Busca dentro del arreglo el objeto cuyo `id` coincida con el parámetro `id` recibido de la URL. Si no existe, devuelve `undefined`.
- `{propiedad ? ( <Card>...</Card> ) : ( <div className="alert alert-warning">... </div> )}`: Renderizado condicional. Si la propiedad fue encontrada, pinta la tarjeta `Card` con su foto y datos; si no existe (o si se entró a `/propiedades` sin elegir un código), muestra el cartel amarillo de advertencia.

PASO 6 Rutas Protegidas con `UserContext` y `<Navigate />`

En aplicaciones reales con cuentas de usuario, existen rutas públicas (`/` , `/propiedades` , `/login`) y **rutas protegidas o privadas** (`/admin`) que solo deben mostrarse si existe una sesión de usuario activa.

6.1. Crear primero las vistas `Administracion.jsx` y `Login.jsx` básicas

Siguiendo nuestro enfoque iterativo, primero creamos ambas vistas con un texto inicial y las probamos en `App.jsx` para confirmar que navegan bien antes de agregarles el contexto de seguridad.

6.2. Crear el contexto de sesión en `src/context/UserContext.jsx`

📄 **src/context/UserContext.jsx (Escribir sin usar el atajo rafc)**

```
import { createContext, useState } from "react";

export const UserContext = createContext();

const UserProvider = ({ children }) => {
  const [user, setUser] = useState(null);

  return (
    <UserContext.Provider value={{ user, setUser }}>
      {children}
    </UserContext.Provider>
  );
};

export default UserProvider;
```

💡 **Dos fundamentos importantes sobre `UserContext.jsx` que aprendimos:**

1. **¿Por qué NO usar el atajo `rafc` en este archivo?** Porque `rafc` crea una función llamada igual que el archivo (`const UserContext = () => {`), abriendo una llave donde no corresponde. En cambio, `export const UserContext = createContext();` NO es una función: es una sola línea que termina en `;` , y debajo de ella va la única función del archivo que es `UserProvider` .
2. **¿Por qué al final ponemos `export default UserProvider;` y no `UserContext` ?** Porque de este archivo necesitan salir **DOS herramientas distintas** hacia afuera y JavaScript solo permite un único `export default` por archivo:
   - `export const UserContext` (línea 3) sale con llaves `{ UserContext }` hacia `App.jsx` , `Login.jsx` y `Administracion.jsx` .
   - `export default UserProvider` (línea 15) sale sin llaves hacia `main.jsx` para envolver la aplicación.

6.3. Envolver `<App />` con `<UserProvider>` en `src/main.jsx`

📄 **src/main.jsx (Versión Definitiva)**

```
import UserProvider from './context/UserContext.jsx'
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import 'bootstrap/dist/css/bootstrap.min.css'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <UserProvider>
      <App />
    </UserProvider>
  </StrictMode>,
)
```

6.4. Proteger la ruta `/admin` en `src/App.jsx` usando `<Navigate />`

📄 **src/App.jsx (Versión Definitiva)**

```
import { useContext } from 'react';
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { UserContext } from './context/UserContext';
import Navbar from './components/Navbar';
import Home from './views/Home';
import Propiedades from './views/Propiedades';
import Administracion from './views/Administracion';
import Login from './views/Login';

function App() {
  const { user } = useContext(UserContext);

  return (
    <div className='App'>
      <BrowserRouter>
        <Navbar />
        <Routes>
          <Route path='/' element={<Home />} />
          <Route path='/propiedades' element={<Propiedades />} />
          <Route path='/propiedades/:id' element={<Propiedades />} />
          <Route
            path='/admin'
            element={user ? <Administracion /> : <Navigate to='/login' />}
          />
          <Route path='/login' element={<Login />} />
        </Routes>
      </BrowserRouter>
    </div>
  )
}

export default App
```

⚠️ **Errores comunes que debemos evitar al conectar el Contexto y `Navigate`:**

- En `main.jsx`: No olvidar escribir `import UserProvider from './context/UserContext.jsx'` arriba si usamos `<UserProvider>` abajo.
- En `App.jsx`: No olvidar agregar `, Navigate` dentro de las llaves de `import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';`.

✅ **Punto de control en pantalla (Paso 6):**

Como `user` inicia en `null`, al hacer clic en **"Administración"** en el menú (o escribir `/admin` en la barra de direcciones), el componente `<Navigate to='/login' />` nos redirige instantáneamente hacia la vista **Login**.

PASO 7 Formulario de Inicio de Sesión y Botón Cerrar Sesión

Para completar el ciclo de rutas protegidas, programamos en `Login.jsx` un formulario donde el corredor ingresa su **Nombre** y **Correo Electrónico**. Al enviarlo, actualizamos el estado global `user` del contexto con `setUser(...)` y redirigimos a `/admin`. Finalmente, en `Administracion.jsx` mostramos los datos del corredor conectado y un botón para **Cerrar Sesión** (`setUser(null)`).

7.1. Código definitivo de `src/views/Login.jsx`

📄 **src/views/Login.jsx (Versión Definitiva con Formulario Controlado)**

```
import { useContext, useState } from "react";
import { useNavigate } from "react-router-dom";
import { UserContext } from "../context/UserContext";

export default function Login() {
  const { user, setUser } = useContext(UserContext);
  const navigate = useNavigate();

  const [nombre, setNombre] = useState("");
  const [email, setEmail] = useState("");

  const iniciarSesion = (e) => {
    e.preventDefault();

    if (nombre.trim() === "" || email.trim() === "") {
      alert("Por favor, ingresa tu nombre y tu correo electrónico.");
      return;
    }

    setUser({
      displayName: nombre,
      email: email,
    });

    navigate("/admin");
  };

  return (
    <div className="mt-5 container">
      <h1>🔑 Acceso a corredores</h1>
      <p className="text-muted">
        Ingresa tus datos para gestionar el catálogo de propiedades.
      </p>

      {user ? (
        <div className="alert alert-success d-inline-block mt-3">
          Ya tienes una sesión activa como <strong>{user.displayName}</strong>
        </div>
      ) : (
        <form
          onSubmit={iniciarSesion}
          className="mx-auto mt-4 p-4 border rounded shadow-sm bg-light"
          style={{ maxWidth: "400px" }}
        >
          <div className="mb-3 text-start">
            <label className="form-label fw-bold">Nombre del Corredor</label>
            <input
              type="text"
              className="form-control"
              placeholder="Ej: Jorge Corredor"
              value={nombre}
```

```
              onChange={({ target }) => setNombre(target.value)}
            />
          </div>

          <div className="mb-3 text-start">
            <label className="form-label fw-bold">Correo Electrónico</label>
            <input
              type="email"
              className="form-control"
              placeholder="Ej: jorge@inmoportal.cl"
              value={email}
              onChange={({ target }) => setEmail(target.value)}
            />
          </div>

          <button type="submit" className="btn btn-primary w-100">
            Iniciar Sesión
          </button>
        </form>
      )}
    </div>
  );
}
```

7.2. Código definitivo de `src/views/Administracion.jsx`

📄 **src/views/Administracion.jsx (Versión Definitiva con Bienvenida y Cerrar Sesión)**

```
import { useContext } from "react";
import { UserContext } from "../context/UserContext";

export default function Administracion() {
  const { user, setUser } = useContext(UserContext);

  const cerrarSesion = () => {
    setUser(null);
  };

  return (
    <div className="mt-5 container">
      <h1>🏢 Panel de Administración Inmobiliaria</h1>
      <p className="text-muted">
        Bienvenido <strong>{user.displayName}</strong> ({user.email})
      </p>

      <button className="btn btn-danger mt-3" onClick={cerrarSesion}>
        Cerrar Sesión
      </button>
    </div>
  );
}
```

⚠️ **Tres cuidados clave que aprendimos en el Paso 7:**

1. **No confundir `useContext` con `UserContext` en los imports:** En la línea 1 se importa el hook `{ useContext } from "react"` (con `u` minúscula) y en la línea 3 se importa nuestro contexto `{ UserContext } from "../context/UserContext"` (con `U` mayúscula y `r`).
2. **Extraer `setUser` (con `s`) y no `getUser`:** En `const { user, setUser } = useContext(UserContext);` debemos usar exactamente los nombres que enviamos en el `value={{ user, setUser }}` del Provider.
3. **Cuidado con el autocompletado de VS Code:** Al escribir `setUser(null)` o `{user.displayName}`, vigilar que el editor no lo cambie por palabras como `SpeechSynthesisUtterance` o `UserContext.displayName`.

🎓 Tabla de Resumen Final para Estudio

Herramienta	De dónde se importa	Cómo se usa	Cuándo ocuparla
`<NavLink>`	`react-router-dom`	`className={({ isActive }) => ...}`	En barras de navegación (`Navbar`) para resaltar visualmente la ruta activa.
`useNavigate()`	`react-router-dom`	`const navigate = useNavigate();` `navigate(`/ruta/${id}`);`	Dentro de funciones (ej: al hacer clic en un botón o enviar un formulario validado).
`useParams()`	`react-router-dom`	`const { id } = useParams();`	En la vista detalle para leer el parámetro dinámico definido con `:id` en el `<Route>`.
`<Navigate />`	`react-router-dom`	`element={user ? <Admin /> : <Navigate to="/login" />}`	En el atributo `element` de un `<Route>` en `App.jsx` para proteger rutas privadas.